

Package ‘GTAPViz’

March 5, 2025

Title Automating GTAP Data Processing and Visualization

Version 0.0.0.9000

Author Pattawee Puangchit <ppuangch@purdue.edu>

Description GTAPViz is an extension package of HARplus, designed to provide seamless data visualization for GTAP model results from ``.HAR`` and ``.SL4`` files. This package focuses on user-friendly visualization tools, enabling GTAP users to analyze and compare simulation results efficiently without requiring deep coding knowledge. With built-in functions for pivoting, mapping, unit assignments, and graphical representation, GTAPViz enhances economic model interpretation by generating structured, publication-ready plots. It is ideal for GTAP researchers, policymakers, and analysts looking for intuitive data exploration and visual insights from GEMPACK outputs.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2

BugReports <https://github.com/bodysbobb/GTAPViz/issues>

URL <https://bodysbobb.github.io/GTAPViz/>

Imports HARplus,

ggplot2,
dplyr,
tidyr,
openxlsx,
colorspace,
grDevices,
rlang,
scales,
purrr,
readxl,
writexl,
utils,
methods,
stringdist,
stats

Remotes github::Bodysbobb/HARplus

Suggests knitr,
rmarkdown,
devtools

VignetteBuilder knitr
Depends R (>= 3.5)

Contents

add_mapping_info	2
comparison_plot	3
convert_units	5
detail_plot	7
get_plot_style_config	9
gtap_macros_data	12
process_gtap_data	13
rename_GTAP_bilateral	16
rename_value	17
report_table	18
stack_plot	19

Index	23
--------------	-----------

add_mapping_info	<i>Add Mapping Information to GTAP Data</i>
------------------	---

Description

Adds descriptions and unit information to GTAP data based on a specified mapping mode. Supports external mappings or default GTAPv7 mappings, allowing users to enrich datasets with standardized metadata.

Usage

```
add_mapping_info(  
  data_list,  
  external_map = NULL,  
  mapping = "GTAPv7",  
  description_info = TRUE,  
  unit_info = TRUE  
)
```

Arguments

data_list	A data structure containing GTAP variables.
external_map	Optional data frame with mapping information (must include "Variable", "Description", and "Unit" columns).
mapping	Character. Mapping mode: • "GTAPv7" (default): Uses standard GTAPv7 definitions. • "Yes": Uses only the supplied descriptions and units from external_map. • "No": Does not add any descriptions or units. • "Mix": Prioritizes external_map, but falls back to GTAPv7 for missing values.

description_info
 Logical. If TRUE, adds description information to the data.

unit_info
 Logical. If TRUE, adds unit information to the data.

Value

A data structure with added mapping information, preserving the original structure.

Author(s)

Pattawee Puangchit

See Also

[convert_units](#), [rename_value](#)

Examples

```
## Not run:
# Add mapping using GTAPv7 defaults
gtap_data <- add_mapping_info(gtap_data, mapping = "GTAPv7")

# Use an external mapping file
gtap_data <- add_mapping_info(gtap_data, external_map = my_mapping, mapping = "Mix")

## End(Not run)
```

comparison_plot

Create Comparative Bar Charts from HAR and SL4 Data

Description

Generates comparative bar charts using GTAP data, allowing multiple visualization options such as panel facets, split grouping, and customizable styles.

Usage

```
comparison_plot(
  data,
  filter_var = NULL,
  x_axis_from,
  split_by = NULL,
  panel_var = "Experiment",
  variable_col = "Variable",
  unit_col = "Unit",
  desc_col = "Description",
  invert_pane = FALSE,
  separate_figure = FALSE,
  var_name_by_description = FALSE,
  add_var_info = FALSE,
  output_dir = NULL,
```

```

width = NULL,
height = NULL,
plot_style_config = NULL
)

```

Arguments

<code>data</code>	A data frame or a list of data frames containing GTAP results.
<code>filter_var</code>	Vector or data frame. If a vector, filters the values in <code>x_axis_from</code> . If a data frame, filters <code>value_col</code> based on matching <code>variable_col</code> values.
<code>x_axis_from</code>	Character. Column name for x-axis categories (e.g., "REG", "Sector").
<code>split_by</code>	Character or vector. Column name(s) for data splitting (e.g., "COMM", "REG", "ACTS"). Set to NULL for no splitting, which is suitable for macro-level analysis (i.e., aggregated values without additional dimensions).
<code>panel_var</code>	Character. Column for panel facets (default: "Experiment").
<code>variable_col</code>	Character. Column containing variable identifiers (default: "Variable").
<code>unit_col</code>	Character. Column containing unit information (default: "Unit").
<code>desc_col</code>	Character. Column containing variable descriptions (default: "Description").
<code>invert_pane</code>	Logical. If TRUE, creates horizontal bars instead of vertical ones (default: FALSE).
<code>separate_figure</code>	Logical. If TRUE, generates separate figures per panel value (default: FALSE).
<code>var_name_by_description</code>	Logical. If TRUE, uses descriptions instead of variable codes in titles (default: FALSE).
<code>add_var_info</code>	Logical. If TRUE, adds the variable code in parentheses after the description (default: FALSE).
<code>output_dir</code>	Character. Directory path to save plots as .png files. If NULL, plots are only returned in R without saving.
<code>width</code>	Numeric. Width of the output figure in inches. If NULL, it is calculated automatically.
<code>height</code>	Numeric. Height of the output figure in inches. If NULL, it is calculated automatically.
<code>plot_style_config</code>	List. Custom style configuration for plots. If NULL, defaults from <code>get_plot_style_config("comparison_plot")</code> are applied.

Details

This function allows for extensive customization of plots through the `plot_style_config` parameter, which integrates ggplot2 configurations. To view and adjust available style options, use: `get_plot_style_config("comparison_plot", as_dataframe = TRUE)`.

Value

A ggplot2 object for a single plot or a list of ggplot2 objects for multiple plots.

Author(s)

Pattawee Puangchit

See Also

[detail_plot](#), [stack_plot](#), [get_plot_style_config](#)

Examples

```
## Not run:
# Basic usage with data frame
p1 <- comparison_plot(
  data = gtap_results,
  x_axis_from = "REG",
  panel_var = "Experiment"
)

# Split by commodity with custom styling
p2 <- comparison_plot(
  data = gtap_results,
  x_axis_from = "REG",
  split_by = "COMM",
  panel_var = "Experiment",
  var_name_by_description = TRUE,
  plot_style_config = list(
    color_tone = "economic",
    title_size = 16,
    show_grid_major_y = TRUE
  )
)

# Save plots to directory with inverted orientation
plots <- comparison_plot(
  data = gtap_results,
  x_axis_from = "REG",
  split_by = "COMM",
  invert_pane = TRUE,
  output_dir = "path/to/output/directory"
)

## End(Not run)
```

convert_units

Convert Units in GTAP Data

Description

Converts values in a dataset to different units based on predefined transformations or custom scaling. Supports manual and automatic conversions for economic and trade-related metrics.

Usage

```
convert_units(
  data,
  change_unit_from = NULL,
  change_unit_to = NULL,
```

```

adjustment = NULL,
value_col = "Value",
unit_col = "Unit",
variable_select = NULL,
variable_col = "Variable",
scale_auto = NULL
)

```

Arguments

<code>data</code>	A data structure (list, data frame, or nested combination).
<code>change_unit_from</code>	Character vector. Units to be converted (case-insensitive).
<code>change_unit_to</code>	Character vector. Target units corresponding to <code>change_unit_from</code> .
<code>adjustment</code>	Character or numeric vector. Specifies conversion operations (e.g., <code>"/1000"</code> to convert million to billion).
<code>value_col</code>	Character. Column name containing values to adjust (default: <code>"Value"</code>).
<code>unit_col</code>	Character. Column name containing unit information (default: <code>"Unit"</code>).
<code>variable_select</code>	Optional character vector. If provided, only these variables are converted.
<code>variable_col</code>	Character. Column name containing variable identifiers (default: <code>"Variable"</code>).
<code>scale_auto</code>	Optional character vector of predefined conversion rules: <code>"mil2bil"</code>: Converts million USD to billion USD (divides by 1000). <code>"bil2mil"</code>: Converts billion USD to million USD (multiplies by 1000). <code>"pct2frac"</code>: Converts percent to fraction (divides by 100). <code>"frac2pct"</code>: Converts fraction to percent (multiplies by 100).

Details

If both `change_unit_from` and `scale_auto` are provided, the function prompts the user to choose between manual and automatic conversion.

Value

A data structure with values converted to the specified units.

Author(s)

Pattawee Puangchit

See Also

[add_mapping_info](#), [rename_value](#)

Examples

```

## Not run:
# Convert million USD to billion USD
gtap_data <- convert_units(gtap_data,
  change_unit_from = "million USD",
  change_unit_to = "billion USD",

```

```

    adjustment = "/1000"
  )

  # Automatic conversion from percent to fraction
  gtap_data <- convert_units(gtap_data, scale_auto = "pct2frac")

  ## End(Not run)

```

detail_plot

Create Comprehensive Detailed Bar Charts for Impact Distribution

Description

Generates detailed bar charts to visualize the distribution of impacts across multiple dimensions, automatically handling multi-dimensional data. The function supports top impact filtering, color coding for positive and negative values, and flexible visualization settings.

Usage

```

detail_plot(
  data,
  filter_var = NULL,
  x_axis_from,
  split_by = NULL,
  panel_var = "Experiment",
  variable_col = "Variable",
  unit_col = "Unit",
  desc_col = "Description",
  var_name_by_description = FALSE,
  add_var_info = FALSE,
  output_dir = NULL,
  top_impact = NULL,
  width = NULL,
  height = NULL,
  separate_figure = FALSE,
  invert_pane = FALSE,
  plot_style_config = NULL
)

```

Arguments

<code>data</code>	A data frame or a list of data frames containing GTAP results.
<code>filter_var</code>	Vector or data frame. If a vector, filters the values in <code>x_axis_from</code> . If a data frame, filters <code>value_col</code> based on matching <code>variable_col</code> values.
<code>x_axis_from</code>	Character. Column name for x-axis categories (e.g., "REG", "Sector").
<code>split_by</code>	Character or vector. Column name(s) for data splitting (e.g., "COMM", "REG", "ACTS"). Set to NULL for no splitting, which is suitable for macro-level analysis (i.e., aggregated values without additional dimensions).
<code>panel_var</code>	Character. Column for panel facets (default: "Experiment").

variable_col	Character. Column containing variable identifiers (default: "Variable").
unit_col	Character. Column containing unit information (default: "Unit").
desc_col	Character. Column containing variable descriptions (default: "Description").
var_name_by_description	Logical. If TRUE, uses descriptions instead of variable codes in titles (default: FALSE).
add_var_info	Logical. If TRUE, adds the variable code in parentheses after the description (default: FALSE).
output_dir	Character. Directory path to save plots as .png files. If NULL, plots are only returned in R without saving.
top_impact	Numeric or NULL. If specified, filters to show only the top N impactful values. NULL shows all values.
width	Numeric. Width of the output figure in inches. If NULL, it is calculated automatically.
height	Numeric. Height of the output figure in inches. If NULL, it is calculated automatically.
separate_figure	Logical. If TRUE, creates a separate figure for each panel value (default: FALSE).
invert_pane	Logical. If TRUE, creates horizontal bars instead of vertical ones (default: FALSE).
plot_style_config	List. Custom style configuration for plots. If NULL, defaults from <code>get_plot_style_config("detail")</code> are applied.

Details

This function allows for extensive customization of plots through the `plot_style_config` parameter, which integrates `ggplot2` configurations. To view and adjust available style options, use: `get_plot_style_config("detail", as_dataframe = TRUE)`.

Value

A `ggplot2` object for a single plot or a list of `ggplot2` objects for multiple plots.

Author(s)

Pattawee Puangchit

See Also

[comparison_plot](#), [stack_plot](#), [get_plot_style_config](#)

Examples

```
## Not run:
# Basic usage showing all impacts
p1 <- detail_plot(
  data = gtap_results,
  x_axis_from = "REG",
  variable_col = "Variable",
  top_impact = NULL
```



```

)

# Show only top 10 impacts (balanced between positive and negative)
p2 <- detail_plot(
  data = gtap_results,
  x_axis_from = "Sector",
  split_by = "Region",
  panel_var = "Experiment",
  variable_col = "Variable",
  top_impact = 10,
  invert_pane = TRUE
)

# Create detailed plots split by commodity
plots <- detail_plot(
  data = gtap_results,
  x_axis_from = "REG",
  split_by = "COMM",
  variable_col = "Variable",
  var_name_by_description = TRUE,
  add_var_info = TRUE,
  output_dir = "path/to/output/directory"
)

## End(Not run)

```

get_plot_style_config *Get Plot Style Configuration*

Description

Returns configuration settings for plot styles, with options to view as a structured dataframe or to look up specific parameters. Also provides parameter validation for custom configurations.

Usage

```

get_plot_style_config(
  plot_type = "comparison",
  parameter_name = NULL,
  show_docs = FALSE,
  validate_custom = NULL,
  as_dataframe = FALSE
)

```

Arguments

plot_type	Character. Type of plot: "comparison" (default), "detail", or "stack".
parameter_name	Character or NULL. Name of specific parameter to return information about.
show_docs	Logical. Whether to include documentation in the output.
validate_custom	List or NULL. Custom configuration settings to validate.
as_dataframe	Logical. Whether to return settings as a dataframe.

Details

This function applies default styling for different types of plots and allows users to customize the appearance. The parameters are grouped as follows:

Title Settings:

- `show_title`: Logical. Show or hide the plot title. Default: `TRUE`
- `title_face`: Character. Font face ("`bold`", "`plain`", "`italic`"). Default: "`bold`"
- `title_size`: Numeric. Font size of title. Default: `20`
- `title_hjust`: Numeric. Horizontal alignment (0 = left, 1 = right). Default: `0.5`
- `add_unit_to_title`: Logical. Append unit to title if applicable. Default: `TRUE`
- `title_margin`: `ggplot2 margin()` object. Default: `ggplot2::margin(10, 0, 10, 0)`
- `title_format`: List. Defines title formatting options (`type = "standard"`, `text = ""`)

X-Axis Settings:

- `show_x_axis_title`: Logical. Show or hide x-axis title. Default: `FALSE`
- `x_axis_title_face`: Character. Font face for x-axis title. Default: "`bold`"
- `x_axis_title_size`: Numeric. Font size of x-axis title. Default: `16`
- `x_axis_title_margin`: `ggplot2 margin()`. Default: `ggplot2::margin(t = 20)`
- `show_x_axis_labels`: Logical. Show or hide x-axis labels. Default: `TRUE`
- `x_axis_text_face`: Character. Font face for x-axis labels. Default: "`bold`"
- `x_axis_text_size`: Numeric. Font size of x-axis labels. Default: `14`
- `x_axis_text_angle`: Numeric. Angle of x-axis labels. Default: `45`
- `x_axis_text_hjust`: Numeric. Horizontal justification of x-axis labels. Default: `1`
- `x_axis_description`: Character. Optional description for the x-axis. Default: `""`

Y-Axis Settings:

- `show_y_axis_title`: Logical. Show or hide y-axis title. Default: `TRUE`
- `y_axis_title_face`: Character. Font face for y-axis title. Default: "`bold`"
- `y_axis_title_size`: Numeric. Font size of y-axis title. Default: `16`
- `y_axis_title_margin`: `ggplot2 margin()`. Default: `ggplot2::margin(r = 20)`
- `show_y_axis_labels`: Logical. Show or hide y-axis labels. Default: `TRUE`
- `y_axis_text_face`: Character. Font face for y-axis labels. Default: "`plain`"
- `y_axis_text_size`: Numeric. Font size of y-axis labels. Default: `14`
- `y_axis_text_angle`: Numeric. Angle of y-axis labels. Default: `0`
- `y_axis_text_hjust`: Numeric. Horizontal justification of y-axis labels. Default: `0`
- `y_axis_description`: Character. Optional description for the y-axis. Default: `""`
- `show_axis_titles_on_all_facets`: Logical. Show axis titles on all facets. Default: `TRUE`

Value Label Settings:

- `show_value_labels`: Logical. Show or hide value labels. Default: `TRUE`
- `value_label_face`: Character. Font face for value labels. Default: "`plain`"
- `value_label_size`: Numeric. Font size of value labels. Default: `5`
- `value_label_position`: Character. Position of value labels ("`above`", "`outside`", "`top`"). Default: "`above`"
- `value_label_decimal_places`: Numeric. Number of decimal places in value labels. Default: `2`

Legend Settings:

- show_legend: Logical. Show or hide legend. Default: FALSE
- show_legend_title: Logical. Show or hide legend title. Default: FALSE
- legend_position: Character. Legend position ("none", "bottom", "right"). Default: "none"
- legend_title_face: Character. Font face for legend title. Default: "bold"
- legend_text_face: Character. Font face for legend text. Default: "plain"
- legend_text_size: Numeric. Font size for legend text. Default: 14

Panel Strip Settings:

- strip_face: Character. Font face for panel strip. Default: "bold"
- strip_text_size: Numeric. Font size for panel strip. Default: 16
- strip_background: Character. Background color of strip. Default: "lightgrey"
- strip_text_margin: ggplot2 margin(). Default: ggplot2::margin(10, 0, 10, 0)

Panel Layout:

- panel_spacing: Numeric. Spacing between panels. Default: 2
- panel_rows: Numeric or NULL. Number of rows in panel layout. Default: NULL
- panel_cols: Numeric or NULL. Number of columns in panel layout. Default: NULL
- theme: ggplot2 theme object or NULL. Custom ggplot theme. Default: NULL

Color Settings:

- color_tone: Character or NULL. Base color theme. Default: NULL
- positive_color: Character. Color for positive values. Default: "#2E8B57"
- negative_color: Character. Color for negative values. Default: "#CD5C5C"
- background_color: Character. Background color of plot. Default: "white"
- grid_color: Character. Color of grid lines. Default: "grey90"
- show_grid_major_x: Logical. Show major grid lines on x-axis. Default: FALSE
- show_grid_major_y: Logical. Show major grid lines on y-axis. Default: TRUE
- show_grid_minor_x: Logical. Show minor grid lines on x-axis. Default: FALSE
- show_grid_minor_y: Logical. Show minor grid lines on y-axis. Default: FALSE

Zero Line Settings:

- show_zero_line: Logical. Show or hide zero line. Default: TRUE
- zero_line_type: Character. Line type ("solid", "dashed", "dotted"). Default: "dashed"
- zero_line_color: Character. Color of zero line. Default: "black"
- zero_line_size: Numeric. Line thickness of zero line. Default: 0.5
- zero_line_position: Numeric. Position of the zero line. Default: 0

Bar Chart Settings:

- bar_width: Numeric. Width of bars. Default: 0.9
- bar_spacing: Numeric. Spacing between bars. Default: 0.9

Scale Expansion Settings:

- expansion_y_mult: Numeric vector. Y-axis expansion. Default: c(0.05, 0.1)
- expansion_x_mult: Numeric vector. X-axis expansion. Default: c(0.05, 0.05)

Value

If `parameter_name` is specified, returns a list with the value and documentation for that parameter. If `validate_custom` is specified, returns the validated configuration list. If `as_dataframe` is `TRUE`, returns a dataframe with configuration settings. Otherwise, returns a list with all configuration settings.

Author(s)

Pattawee Puangchit

See Also

[comparison_plot](#), [detail_plot](#), [stack_plot](#)

Examples

```
# Get default configuration
config <- get_plot_style_config("comparison")

# Get information about a specific parameter
param_info <- get_plot_style_config("detail", "bar_width")

# Get as structured dataframe
config_df <- get_plot_style_config("stack", as_dataframe = TRUE)

# Validate custom configuration
custom_config <- list(title_size = 24, bar_width = 0.7)
validated <- get_plot_style_config("comparison", validate_custom = custom_config)
```

gtap_macros_data

Extract and Aggregate Scalar Macroeconomic Variables

Description

Extracts scalar macroeconomic variables from multiple SL4 datasets and aggregates them into a structured data frame.

Usage

```
gtap_macros_data(
  input_dir = NULL,
  experiment_names = NULL,
  output_dir = NULL,
  output_formats = NULL,
  subtotal_level = FALSE,
  select_var = NULL
)
```

Arguments

`input_dir` Character. Directory containing SL4 files.

`experiment_names` Character vector. List of experiment names corresponding to SL4 files.

`output_dir` Character (optional). Directory to save exported data.

`output_formats` Character vector (optional). List of output formats (e.g., "csv", "xlsx").

`subtotal_level` Logical. Whether to include subtotal levels in the processed data.

`select_var` Character vector (optional). List of specific variable names to filter from the final result. If NULL, all variables are returned.

Value

A sorted dataframe containing processed GTAP macro data.

Author(s)

Pattawee Puangchit

See Also

[add_mapping_info](#), [process_gtap_data](#)

Examples

```
## Not run:
# Method 1: Extract all variables
input_dir <- paste0(input.folder)
experiment <- c("EXP1", "EXP2") # File name (.sl4)
Macros <- gtap_macros_data(input_dir, experiment_names = experiment,
  subtotal_level = FALSE)

# Method 2: Filter specific variables
Macros <- gtap_macros_data(input_dir, experiment_names = experiment,
  select_var = c("qgdp", "pop", "gdpxp"))

## End(Not run)
```

Description

Processes GTAP data from SL4 and HAR files with options for exporting and preparing plot-ready data.

Usage

```

process_gtap_data(
  experiment,
  mapping_info = "GTAPv7",
  project_dir = NULL,
  input_dir = NULL,
  output_dir = NULL,
  output_formats = NULL,
  plot_data = FALSE,
  region_select = NULL,
  sector_select = NULL,
  subtotal = FALSE,
  sl4_list_name = "sl4.plot.data",
  har_list_name = "har.plot.data",
  sl4_structure_name = "sl4.structure",
  har_structure_name = "har.structure",
  GTAPMacro_name = "GTAPMacro",
  sl4var = NULL,
  harvar = NULL,
  sl4map = NULL,
  harmap = NULL
)

```

Arguments

experiment	Character vector. Case names to process.
mapping_info	Character. Mapping mode: "GTAPv7" (default), "Yes", "No", or "Mix".
project_dir	Character. Path to the project folder with "in" and "out" subfolders.
input_dir	Character. Path to the input folder. Overrides project_dir/in if specified.
output_dir	Character. Path to the output folder. Overrides project_dir/out if specified.
output_formats	Character vector or list. Exports data in these formats (valid: "csv", "stata", "rds", "txt").
plot_data	Logical. If TRUE, prepares data for plotting and assigns to variables.
region_select	Optional vector. Specifies regions to filter the data.
sector_select	Optional vector. Specifies sectors to filter the data.
subtotal	Logical. If TRUE, includes subtotal data. Default is FALSE.
sl4_list_name	Character. Variable name for SL4 plotting data if generating plot data. Default is "sl4.plot.data".
har_list_name	Character. Variable name for HAR plotting data if generating plot data. Default is "har.plot.data".
sl4_structure_name	Character. Variable name for SL4 structure if generating plot data. Default is "sl4.structure".
har_structure_name	Character. Variable name for HAR structure if generating plot data. Default is "har.structure".
GTAPMacro_name	Character. Variable name for GTAP macro data if generating plot data. Default is "GTAPMacro".

sl4var	Data frame, NULL, or FALSE. Variables to extract from SL4 files. Set to NULL to extract all variables, or FALSE to skip SL4 processing.
harvar	Data frame, NULL, or FALSE. Variables to extract from HAR files. Set to NULL to extract all variables, or FALSE to skip HAR processing.
sl4map	Data frame or NULL. Mapping information for SL4 variables (with "Variable", "Description", and "Unit" columns).
harmap	Data frame or NULL. Mapping information for HAR variables (with "Variable", "Description", and "Unit" columns).

Details

This function provides a complete automation workflow for processing GTAP model outputs, with flexible output options and optional filtering for plot data.

The key parameters `sl4var` and `harvar` each accept three different input types:

- **Data frame:** Contains variable mappings with required "Variable" column. When provided, only specified variables will be extracted.
- **NULL:** Extracts all available variables from the respective file type.
- **FALSE:** Completely skips processing of that file type, allowing the function to focus only on the other file type.

The `mapping_info` parameter controls how descriptions and units are assigned:

- **GTAPv7:** Uses standard GTAPv7 definitions (default)
- **Yes:** Uses only the supplied descriptions and units from `sl4map/harmap`
- **No:** Does not add any descriptions or units
- **Mix:** Prioritizes supplied descriptions and units, falling back to GTAPv7 for any missing values

Value

Returns the processed data invisibly, which will not be printed to the console.

Author(s)

Pattawee Puangchit

See Also

[add_mapping_info](#), [gtap_macros_data](#)

Examples

```
## Not run:
# Extract data with region and experiment filters
process_gtap_data(
  sl4var = NULL,
  harvar = NULL,
  sl4map = sl4map,
  harmap = harmap,
  region_select = selected_regions,
  sector_select = NULL,
  subtotal = FALSE,
```

```

experiment = experiment,
mapping_info = info.mode,
project_dir = project.folder,
plot_data = TRUE,
output_formats = list(
  "csv" = "No",
  "stata" = "No",
  "rds" = "No",
  "txt" = "No"))

## End(Not run)

```

rename_GTAP_bilateral *Rename GTAP Bilateral Trade Columns*

Description

Renames bilateral trade columns in GTAP data to standardized names, ensuring consistency in regional trade flows.

Usage

```
rename_GTAP_bilateral(data)
```

Arguments

data Data structure containing GTAP bilateral trade data.

Value

The same data structure with renamed bilateral trade columns.

Author(s)

Pattawee Puangchit

See Also

[add_mapping_info](#), [convert_units](#)

Examples

```

## Not run:
# Rename bilateral trade columns in a GTAP dataset
gtap_data <- rename_GTAP_bilateral(gtap_data)

## End(Not run)

```

rename_value	<i>Rename Values in a Column</i>
--------------	----------------------------------

Description

Replaces specific values in a column based on a provided mapping file. Supports renaming across nested data structures and preserves factor levels.

Usage

```
rename_value(data, column_name = NULL, mapping.file)
```

Arguments

data	Data structure (data frame, list, or nested combination).
column_name	Character. Column to modify. If NULL, the function extracts it from mapping.file.
mapping.file	Data frame with "OldName" and "NewName" columns for renaming.

Value

The same data structure with specified values replaced.

Author(s)

Pattawee Puangchit

See Also

[add_mapping_info](#), [convert_units](#)

Examples

```
## Not run:  
# Rename variables in a dataset  
rename_map <- data.frame(OldName = c("old_var1", "old_var2"), NewName = c("new_var1", "new_var2"))  
gtap_data <- rename_value(gtap_data, column_name = "Variable", mapping.file = rename_map)  
  
## End(Not run)
```

report_table

*Generate a Detailed Report Table***Description**

Transforms one or more datasets into a wide-format table based on specified grouping columns and pivot column(s). Supports subtotal filtering, renaming, and exporting to Excel.

Usage

```
report_table(
  data_list,
  pivot_col,
  total_column = FALSE,
  export_table = FALSE,
  separate_file = FALSE,
  output_dir = NULL,
  sheet_names = NULL,
  include_units = FALSE,
  component_exclude = NULL,
  group_by = NULL,
  rename_cols = NULL,
  var_name_by_description = TRUE,
  add_var_info = FALSE,
  decimal = 2,
  unit_select = NULL,
  separate_sheet_by = NULL,
  subtotal_level = FALSE,
  repeat_label = FALSE,
  workbook_name = "detail_results",
  add_group_line = FALSE
)
```

Arguments

data_list	A named list of data frames to be processed.
pivot_col	A named list specifying which column is pivoted into wide format for each dataset.
total_column	Logical. If TRUE, includes a "Total" column summing numeric values.
export_table	Logical. If TRUE, exports the tables to an Excel file.
separate_file	Logical. If TRUE, saves each dataset in a separate Excel file.
output_dir	Character. Directory where Excel files are written if export_table = TRUE.
sheet_names	Optional named list for custom sheet names.
include_units	Logical. If TRUE, includes the "Unit" column in grouping.
component_exclude	Optional character vector specifying pivoted values to exclude.
group_by	A list or character vector of grouping columns, or NULL for automatic detection.
rename_cols	A named list for renaming columns.

var_name_by_description	Logical. If TRUE, replaces Variable names with their Description.
add_var_info	Logical. If TRUE, appends the Variable code in parentheses after the Description.
decimal	Numeric. Number of decimal places for rounding values.
unit_select	Optional. Filters rows based on the specified unit.
separate_sheet_by	Optional column name to split sheets in Excel.
subtotal_level	Logical. If TRUE, includes all subtotal values; otherwise, keeps only Subtotal = "TOTAL".
repeat_label	Logical. If TRUE, repeats the first group column in exports for clarity.
workbook_name	Character. Name of the Excel workbook (without extension).
add_group_line	Logical. If TRUE, adds a thin line after each group in the exported table.

Value

A named list of transformed data frames. If export_table = TRUE, tables are saved as Excel files.

Author(s)

Pattawee Puangchit

See Also

[add_mapping_info](#), [convert_units](#), [rename_value](#)

Examples

```
## Not run:
# Create a detailed report table with wide-format pivoting
report_data <- report_table(
  data_list = my_data,
  pivot_col = list("Variable" = "Value"),
  export_table = TRUE,
  output_dir = "results/"
)

## End(Not run)
```

stack_plot	Create Comprehensive Stacked Bar Charts for Decomposition Analysis
------------	--

Description

Generates stacked bar charts to visualize the composition of values across multiple dimensions, automatically handling multi-dimensional data. The function supports both stacked and unstacked presentations, making it particularly useful for decomposition analysis.

Usage

```

stack_plot(
  data,
  filter_var = NULL,
  x_axis_from,
  stack_value_from,
  split_by = NULL,
  panel_var = "Experiment",
  variable_col = "Variable",
  unit_col = "Unit",
  desc_col = "Description",
  invert_pane = FALSE,
  separate_figure = FALSE,
  unstack_plot = FALSE,
  output_dir = NULL,
  var_name_by_description = FALSE,
  add_var_info = FALSE,
  show_total = TRUE,
  y_limit = NULL,
  width = NULL,
  height = NULL,
  top_impact = NULL,
  plot_style_config = NULL
)

```

Arguments

<code>data</code>	A data frame or a list of data frames containing GTAP results.
<code>filter_var</code>	Vector or data frame. If a vector, filters the values in <code>x_axis_from</code> . If a data frame, filters <code>value_col</code> based on matching <code>variable_col</code> values.
<code>x_axis_from</code>	Character. Name of the column to use for x-axis categories (e.g., "REG", "Sector").
<code>stack_value_from</code>	Character. Name of the column containing stack component categories (e.g., "COMM" for commodities).
<code>split_by</code>	Character or vector. Column name(s) for data splitting (e.g., "COMM", "REG", "ACTS"). Set to NULL for no splitting, which is suitable for macro-level analysis (i.e., aggregated values without additional dimensions).
<code>panel_var</code>	Character. Column for panel facets (default: "Experiment").
<code>variable_col</code>	Character. Column containing variable identifiers (default: "Variable").
<code>unit_col</code>	Character. Column containing unit information (default: "Unit").
<code>desc_col</code>	Character. Column containing variable descriptions (default: "Description").
<code>invert_pane</code>	Logical. If TRUE, creates horizontal bars instead of vertical ones (default: FALSE).
<code>separate_figure</code>	Logical. If TRUE, creates a separate figure for each panel value (default: FALSE).
<code>unstack_plot</code>	Logical. If TRUE, creates separate bar plots for each <code>x_axis_from</code> value instead of stacked bars (default: FALSE).

output_dir	Character. Directory path to save plots as .png files. If NULL, plots are only returned in R without saving.
var_name_by_description	Logical. If TRUE, uses descriptions instead of variable codes in titles (default: FALSE).
add_var_info	Logical. If TRUE, adds the variable code in parentheses after the description (default: FALSE).
show_total	Logical. If TRUE, displays total values above stacked bars (default: TRUE).
y_limit	Numeric vector. Manual limits for the y-axis. If NULL, it is calculated automatically.
width	Numeric. Width of the output figure in inches. If NULL, it is calculated automatically.
height	Numeric. Height of the output figure in inches. If NULL, it is calculated automatically.
top_impact	Numeric or NULL. If specified, filters to show only the top N impactful values. NULL shows all values.
plot_style_config	List. Custom style configuration for plots. If NULL, defaults from <code>get_plot_style_config("stack")</code> are applied.

Details

This function allows for extensive customization of plots through the `plot_style_config` parameter, which integrates `ggplot2` configurations. To view and adjust available style options, use: `get_plot_style_config("stack", as_dataframe = TRUE)`.

Value

A `ggplot2` object for a single plot or a list of `ggplot2` objects for multiple plots.

Author(s)

Pattawee Puangchit

See Also

[detail_plot](#), [comparison_plot](#), [get_plot_style_config](#)

Examples

```
## Not run:
# Basic stacked bar chart by commodity
p1 <- stack_plot(
  data = gtap_results,
  x_axis_from = "REG",
  stack_value_from = "COMM"
)

# Welfare Decomposition
p2 <- stack_plot(
  data = headerA,
  x_axis_from = "Region",
  stack_value_from = "COLUMN",
```

```
split_by = FALSE,
unstack_plot = TRUE,
show_total = TRUE,

)

# Terms of trade decomposition
plots <- stack_plot(
  data = headerE1,
  x_axis_from = "Commodity",
  stack_value_from = "PRICES",
  split_by = "Region",
  top_impact = 10,
  invert_pane = TRUE,
  output_dir = "path/to/output/directory",
  plot_style_config = list(
    color_tone = "economic",
    show_legend = TRUE,
    legend_position = "bottom"
  )
)

## End(Not run)
```

Index

`add_mapping_info`, [2](#), [6](#), [13](#), [15–17](#), [19](#)

`comparison_plot`, [3](#), [8](#), [12](#), [21](#)

`convert_units`, [3](#), [5](#), [16](#), [17](#), [19](#)

`detail_plot`, [5](#), [7](#), [12](#), [21](#)

`get_plot_style_config`, [5](#), [8](#), [9](#), [21](#)

`gtap_macros_data`, [12](#), [15](#)

`process_gtap_data`, [13](#), [13](#)

`rename_GTAP_bilateral`, [16](#)

`rename_value`, [3](#), [6](#), [17](#), [19](#)

`report_table`, [18](#)

`stack_plot`, [5](#), [8](#), [12](#), [19](#)